

Data Collection Milestone and Acceleration Scoping

Date: 18 July 2026

Test vehicle: Maruti Suzuki Fronx

1. Aim of Today's Work

Today's aim was to prove the complete data collection pipeline working end to end on a real vehicle. This means the Raspberry Pi reading live engine data from the car's OBD-II port over Bluetooth, reading live position from the GPS module at the same time, combining both into a single structured packet, and sending that packet over the network to a receiver that decodes and displays it. Earlier days proved each of these pieces on its own. Today was the first time all of them ran together on the car. Alongside this, we also scoped the next phase of the project, which is automated acceleration, and that work is reported in the later sections.

2. System Setup

The Raspberry Pi 4 connects to the car's OBD-II port through an ELM327 Bluetooth adapter, which gives it access to the engine's live sensor data. A NEO-6M GPS module is wired to the Pi's serial pins and provides live latitude, longitude, satellite count and fix status. The Pi runs a single Python program (`elm327_bt.py`) that manages both inputs, builds the combined packet, writes every reading to a CSV log on the Pi, and forwards each packet over the network to a receiver. In this test the receiver is a laptop, which stands in for the cloud server that will receive the data in the final system.

3. Data Acquisition on the Raspberry Pi

When the program starts it opens the GPS serial port and the Bluetooth link, then runs the standard ELM327 initialisation sequence (`ATZ`, `ATE0`, `ATL0`, `ATSP0`, `ATH0`), with each step confirmed by an OK reply. It then asks the car which sensors it supports and begins a capture loop that runs once per second. Each cycle it requests each sensor in turn, shown as the [RAW] lines (for example `010C sent, 41 0C 0D 2E` returned for engine RPM), reads the GPS, and prints a combined summary. The screenshot below shows a clean start on the car, ending with the first live reading: RPM 843.5, coolant 88 degrees, battery 13.8V, and GPS locked with 6 satellites.

```
crw-rw---- 1 root dialout 216, 0 Jul 18 16:36 /dev/rfcomm0
lus-pi@LUS-PI:~/Car-Automu x + v
lus-pi@LUS-PI:~/Car-Automation/backend $ python3 elm327_bt.py --raw
=====
ELM327 Bluetooth OBD-II Acquisition Script
COM Port : /dev/rfcomm0 | Baud: 38400
Log Dir : /home/lus-pi/Car-Automation/logs
Backend : DISABLED (offline mode)
TCP : DISABLED
Modes : stream=False, tcp=False, raw=True, fast=False
=====
[LOG] Session logs -> /home/lus-pi/Car-Automation/logs
[LOG] Session 163712 started.
[GPS] Opened /dev/serial0 at 9600 baud.
[BT] Opened /dev/rfcomm0 at 38400 baud.
[BT] Initialising ELM327...
[BT] ATZ -> OK
[BT] ATE0 -> OK
[BT] ATL0 -> OK
[BT] ATSP0 -> OK
[BT] ATH0 -> OK
[BT] ELM327 initialised successfully.
[OBD] === SUPPORTED PIDS (0100) ===
[OBD] 'SEARCHING... \r41 00 0E 3F B0 13'
[OBD] =====
[LOG] Created /home/lus-pi/Car-Automation/logs/2026-07-18/stream/stream_2026-07-18.csv
[OBD] Starting synchronous capture loop (1 cycle per 1.0s)...
[OBD] --raw modifier enabled: printing raw hex bytes
[RAW] 010C -> 41 0C 0D 2E
[LOG] Created /home/lus-pi/Car-Automation/logs/2026-07-18/raw/raw_2026-07-18.csv
[RAW] 010D -> 41 0D 00
[RAW] 0105 -> 41 05 80
[RAW] 0104 -> 41 04 6D
[RAW] 0111 -> 41 11 28
[RAW] 012F -> 7F 01 2F
[RAW] 0110 -> 41 10 01 07
[RAW] 010F -> 41 0F 65
[RAW] 0146 -> 7F 01 46
[RAW] 010B -> 41 0B 33
[RAW] 0101 -> 41 01 00 07 E1 00
[OBD] RPM=843.5 spd=0km/h cool=88C fuel=None% maf=2.63 batt=13.8V
[GPS] lat=12.805346 lng=79.938221 sats=6 fix=True
```

Raspberry Pi: ELM327 initialisation and first combined OBD and GPS reading on the vehicle.

For each cycle the program prints three lines: an [OBD] line with the decoded engine values, a [GPS] line with position, and a [PACKET] line showing the exact 32 bytes that will be logged and sent. The screenshot below shows several complete cycles, including the raw request and response for every sensor.

```
lus-pi@LUS-PI:~/Car-Automu x + v
[OBD] RPM=1027.0 spd=0km/h cool=87C fuel=None% maf=3.49 batt=13.8V
[GPS] lat=12.805332 lng=79.938206 sats=6 fix=True
[PACKET] 6A 5B 5E 8F 10 0C 00 00 C3 64 D4 04 C3 C2 9E 46 01 5D 00 1F 7F FF 8A 00 00 41 65 FF 37 C0 25 6F
[RAW] 010C -> 41 0C 0D AE
[RAW] 010D -> 41 0D 00
[RAW] 0105 -> 41 05 7F
[RAW] 0104 -> 41 04 6D
[RAW] 0111 -> 41 11 28
[RAW] 012F -> 7F 01 2F
[RAW] 0110 -> 41 10 01 0A
[RAW] 010F -> 41 0F 65
[RAW] 0146 -> 7F 01 46
[RAW] 010B -> 41 0B 32
[RAW] 0101 -> 41 01 00 07 E1 00
[OBD] RPM=875.5 spd=0km/h cool=87C fuel=None% maf=2.66 batt=13.8V
[GPS] lat=12.805332 lng=79.938207 sats=6 fix=True
[PACKET] 6A 5B 5E 90 0D AE 00 00 C3 64 D4 04 C3 C2 9F 46 01 0A 00 1F 7F FF 8A 00 00 55 65 FF 32 C0 26 E1
[RAW] 010C -> 41 0C 0D 80
[RAW] 010D -> 41 0D 00
[RAW] 0105 -> 41 05 7F
[RAW] 0104 -> 41 04 6D
[RAW] 0111 -> 41 11 28
[RAW] 012F -> 7F 01 2F
[RAW] 0110 -> 41 10 01 0D
[RAW] 010F -> 41 0F 65
[RAW] 0146 -> 7F 01 46
[RAW] 010B -> 41 0B 32
[RAW] 0101 -> 41 01 00 07 E1 00
[OBD] RPM=864.0 spd=0km/h cool=87C fuel=None% maf=2.69 batt=13.8V
[GPS] lat=12.805333 lng=79.938208 sats=7 fix=True
[PACKET] 6A 5B 5E 91 0D 80 00 00 C3 64 D5 04 C3 C2 A0 47 01 0D 00 1F 7F FF 8A 00 00 54 65 FF 32 C0 27 41
[RAW] 010C -> 41 0C 0D 70
[RAW] 010D -> 41 0D 00
[RAW] 0105 -> 41 05 7F
[RAW] 0104 -> 41 04 6D
[RAW] 0111 -> 41 11 28
[RAW] 012F -> 7F 01 2F
[RAW] 0110 -> 41 10 01 10
[RAW] 010F -> 41 0F 65
[RAW] 0146 -> 7F 01 46
```

Raspberry Pi: decoded OBD values, GPS position, and the packed 32-byte packet for each one-second cycle.

4. How the Data Is Structured

Each reading is packed into a fixed 32-byte packet before it is logged or sent. Keeping every packet the same fixed size makes the data easy to store, transmit and decode

reliably, and lets the receiver check each packet for corruption. The table below shows what each byte holds.

Bytes	Field	How it is stored
0 to 3	Timestamp	Millisecond counter
4 to 5	Engine RPM	Value x 4
6	Vehicle speed	km/h
7 to 10	GPS latitude	Signed, x 1,000,000
11 to 14	GPS longitude	Signed, x 1,000,000
15	GPS satellites and fix flag	Packed into one byte
16 to 17	Air flow (MAF)	g/s x 100
18	Gear (estimated)	Derived from RPM and speed
19	Throttle position	Percent x 2
20	Coolant temperature	Degrees C + 40
21	Fuel level	Percent (0xFF if unsupported)
22	Battery voltage	Volts x 10
23	Warning light and fault count	Packed into one byte
24	AC / brake / clutch flags	Bit-packed with validity bits
25	Engine load	Percent x 2
26	Intake air temperature	Degrees C + 40
27	Ambient temperature	Degrees C + 40 (0xFF if unsupported)
28	Manifold pressure (MAP)	kPa
29	Device health	Status byte
30	Sequence counter	Rolls 0 to 255
31	CRC-8 checksum	Integrity check over the packet

Two points are worth noting. First, values are scaled so they fit neatly into whole bytes and are unpacked back into real units by the receiver. For example RPM is stored as the value multiplied by four, and temperatures are stored with an offset of forty. Second, the final byte is a CRC-8 checksum calculated across the whole packet. The receiver recalculates it on arrival and flags any packet where it does not match, so corrupted data is caught rather than trusted.

5. Sending the Data to the Laptop and Cloud

With the pipeline proven on the Pi alone, the same program was run again with network forwarding enabled. The banner confirms that raw packet forwarding to the receiver on port 9000 is active. Each packet is now sent over the network as it is captured, while still being logged locally on the Pi at the same time. Because the packets are already sent over a standard network connection, the same path can send

the data straight to a real cloud endpoint once we point it at one, with no change to how the data is captured or packed. The laptop here simply stands in for that cloud server.

```
remote: Resolving deltas: 100% (5/5), completed with 3 local objects.
To https://github.com/IoT-Internship-CarAutomation-LUS/Car-Automation.git
8720044..3e91a84  gps-integration -> gps-integration
lus-pi@LUS-PI:~/Car-Automation $ scp lus-pi@pi-ip:~/Car-Automation/backend/decoded_receiver.py .
-bash: pi-ip: No such file or directory
lus-pi@LUS-PI:~/Car-Automation $ hostname -I
10.20.126.232
lus-pi@LUS-PI:~/Car-Automation $ cd ~/Car-Automation/backend
python3 elm327_bt.py --raw --tcp 10.20.126.107
=====
ELM327 Bluetooth OBD-II Acquisition Script
COM Port : /dev/rfcomm0 | Baud: 38400
Log Dir : /home/lus-pi/Car-Automation/logs
Backend : DISABLED (offline mode)
TCP : 10.20.126.107:9000
Modes : stream=False, tcp=True, raw=True, fast=False
=====
[LOG] Session logs -> /home/lus-pi/Car-Automation/logs
[LOG] Session 164349 started.
[TCP] --tcp enabled. Will forward raw packets best-effort to 10.20.126.107:9000
[GPS] Opened /dev/serial0 at 9600 baud.
[BT] Opened /dev/rfcomm0 at 38400 baud.
[BT] Initialising ELM327...
[BT] ATZ -> OK
[BT] ATE0 -> OK
[BT] ATL0 -> OK
[BT] ATSP0 -> OK
[BT] ATH0 -> OK
[BT] ELM327 initialised successfully.
[OBD] ===== SUPPORTED PIDS (0100) =====
[OBD] 'SEARCHING...r\41 00 BE 3F B8 13'
[OBD] =====
[LOG] Appending to /home/lus-pi/Car-Automation/logs/2026-07-18/stream/stream_2026-07-18.csv
[OBD] Starting synchronous capture loop (1 cycle per 1.0s)...
[OBD] --raw modifier enabled: printing raw hex bytes
[OBD] --tcp modifier enabled: raw packet forwarding to 10.20.126.107:9000 active
[RAW] 010C -> 41 0C 0D 28
[LOG] Appending to /home/lus-pi/Car-Automation/logs/2026-07-18/raw/raw_2026-07-18.csv
[RAW] 0100 -> 41 8D 09
[RAW] 0105 -> 41 05 81
[RAW] 0104 -> 41 04 6E
```

Raspberry Pi: the same capture running with live packet forwarding enabled.

6. Data Received and Decoded on the Laptop

On the receiving machine, the program listens for the connection, accepts it from the Pi, and prints each packet as it arrives after unpacking and checking it. Every line shows real decoded values with `fix=True` and `crc_valid=True`, which confirms the packet arrived intact and passed its integrity check. This is the first time the full path, from the car through the Pi to a separate machine, was shown working end to end.

```
C:\Users\shahi>python decoded_receiver.py
[TCPIP] decoded_receiver listening on 0.0.0.0:9000
[TCPIP] Waiting for a connection...
[TCPIP] Connected: 10.20.126.232:50542
RPM=842.0 speed=0 coolant=89 fuel=None lat=12.805345 lng=79.938192 fix=True crc_valid=True
RPM=861.5 speed=0 coolant=89 fuel=None lat=12.805345 lng=79.938192 fix=True crc_valid=True
RPM=840.5 speed=0 coolant=89 fuel=None lat=12.805344 lng=79.938192 fix=True crc_valid=True
RPM=851.5 speed=0 coolant=89 fuel=None lat=12.805344 lng=79.938193 fix=True crc_valid=True
RPM=847.0 speed=0 coolant=89 fuel=None lat=12.805344 lng=79.938193 fix=True crc_valid=True
RPM=838.0 speed=0 coolant=89 fuel=None lat=12.805344 lng=79.938193 fix=True crc_valid=True
RPM=856.0 speed=0 coolant=89 fuel=None lat=12.805344 lng=79.938193 fix=True crc_valid=True
RPM=837.0 speed=0 coolant=89 fuel=None lat=12.805345 lng=79.938193 fix=True crc_valid=True
RPM=850.5 speed=0 coolant=89 fuel=None lat=12.805345 lng=79.938193 fix=True crc_valid=True
RPM=841.0 speed=0 coolant=89 fuel=None lat=12.805346 lng=79.938192 fix=True crc_valid=True
RPM=841.5 speed=0 coolant=89 fuel=None lat=12.805347 lng=79.938192 fix=True crc_valid=True
RPM=861.5 speed=0 coolant=89 fuel=None lat=12.805348 lng=79.938192 fix=True crc_valid=True
RPM=841.0 speed=0 coolant=89 fuel=None lat=12.805349 lng=79.938192 fix=True crc_valid=True
RPM=854.5 speed=0 coolant=89 fuel=None lat=12.80535 lng=79.938191 fix=True crc_valid=True
RPM=845.5 speed=0 coolant=89 fuel=None lat=12.80535 lng=79.938191 fix=True crc_valid=True
RPM=851.0 speed=0 coolant=89 fuel=None lat=12.805351 lng=79.938191 fix=True crc_valid=True
RPM=855.0 speed=0 coolant=89 fuel=None lat=12.805352 lng=79.938191 fix=True crc_valid=True
RPM=838.5 speed=0 coolant=89 fuel=None lat=12.805354 lng=79.938191 fix=True crc_valid=True
RPM=856.5 speed=0 coolant=89 fuel=None lat=12.805355 lng=79.938191 fix=True crc_valid=True
RPM=849.5 speed=0 coolant=89 fuel=None lat=12.805356 lng=79.938191 fix=True crc_valid=True
RPM=857.0 speed=0 coolant=89 fuel=None lat=12.805358 lng=79.938191 fix=True crc_valid=True
RPM=845.0 speed=0 coolant=89 fuel=None lat=12.805359 lng=79.938191 fix=True crc_valid=True
RPM=850.0 speed=0 coolant=89 fuel=None lat=12.805359 lng=79.938191 fix=True crc_valid=True
RPM=858.5 speed=0 coolant=89 fuel=None lat=12.805359 lng=79.938191 fix=True crc_valid=True
RPM=840.5 speed=0 coolant=89 fuel=None lat=12.805361 lng=79.938191 fix=True crc_valid=True
RPM=851.5 speed=0 coolant=89 fuel=None lat=12.805362 lng=79.938189 fix=True crc_valid=True
RPM=849.0 speed=0 coolant=89 fuel=None lat=12.805362 lng=79.938189 fix=True crc_valid=True
RPM=849.5 speed=0 coolant=89 fuel=None lat=12.805364 lng=79.938188 fix=True crc_valid=True
RPM=857.5 speed=0 coolant=89 fuel=None lat=12.805364 lng=79.938187 fix=True crc_valid=True
RPM=849.5 speed=0 coolant=89 fuel=None lat=12.805366 lng=79.938186 fix=True crc_valid=True
RPM=926.0 speed=0 coolant=89 fuel=None lat=12.805368 lng=79.938185 fix=True crc_valid=True
RPM=935.0 speed=0 coolant=89 fuel=None lat=12.805369 lng=79.938184 fix=True crc_valid=True
RPM=840.0 speed=0 coolant=89 fuel=None lat=12.805369 lng=79.938184 fix=True crc_valid=True
RPM=845.0 speed=0 coolant=89 fuel=None lat=12.805369 lng=79.938184 fix=True crc_valid=True
RPM=833.0 speed=0 coolant=89 fuel=None lat=12.80537 lng=79.938183 fix=True crc_valid=True
```

Laptop (standing in for the cloud): packets received from the Pi, decoded, every packet passing its CRC check.

7. What the Captured Data Shows

Across two capture sessions this afternoon the system logged 285 complete packets, and every one of them passed its CRC check. Engine RPM ranged from about 688 to 3126, with the higher figure recorded during a deliberate throttle press to confirm that the readings track the engine in real time. The air-flow reading (MAF) rose and fell together with the RPM as expected. Coolant temperature held between 87 and 90 degrees, battery voltage read a steady 13.8V, and engine load, throttle position, intake temperature and manifold pressure all reported sensible values throughout. The GPS held a fix on every single packet, with 6 to 7 satellites and a stable position, which is the expected result for a stationary vehicle. Vehicle speed read zero throughout, as the car was kept stationary and in neutral for this session. Capturing data while the vehicle is moving is the next step.

8. Signals the Vehicle Does Not Report, and How They Can Be Recovered

Two of the requested values came back marked as unsupported on every cycle: fuel level and ambient air temperature. For these, the car answers with a not-supported code (7F) rather than a value, which the software correctly records as blank rather than as zero. This is a characteristic of this particular vehicle and not a fault in the system.

Some other signals we want, such as brake, clutch and air-conditioning status, are not exposed as standard OBD values at all on these cars. These can still be recovered, but by a different method called CAN sniffing. A CAN module is connected to the car's data bus in listen-only mode, and while it records, we operate the control we are interested in, for example pressing the brake, and watch which message on the bus changes in response. That change identifies the signal. Once found, it can be added as a new field in the 32-byte packet, exactly like the values we already decode.

This is more involved than reading standard OBD values. The messages are not documented and differ from one car model to another, so each signal has to be found by observation on the specific vehicle. We began looking at this on the Fronx today, but chose to first lock down the complete ELM327 and GPS data, which is the larger and more reliable part, before spending time on sniffing. We did not have enough time to finish sniffing and decoding these signals today. With more time, we expect to identify and add them over the coming week.

9. Acceleration: What We Scoped

Alongside data collection, we scoped how the car could be made to accelerate automatically, which is the goal of the next phase. There are two possible routes, and we looked at both.

The first route is electronic: sending an acceleration command to the car through the OBD port, or injecting a signal onto the CAN bus that imitates the accelerator pedal. After investigating this, our finding is that it is not workable in practice. The OBD port is a read-only diagnostic interface and does not accept commands that drive the throttle. The engine control unit is designed to reject external or spoofed pedal signals for safety reasons, so the car cannot be driven this way from a Raspberry Pi or any other outside electronics. We could keep trying through the OBD port, but based on what we found it is very unlikely to work and would take a long time for a low chance of success.

The second route is physical: mounting a linear actuator against the accelerator pedal and driving it under microcontroller control, so the pedal is pressed by a precise, repeatable amount to move the car smoothly and in a controlled, automated way. This is the more realistic path, and it is the one we recommend taking forward.

If the team has any documentation, contacts or reference material on driving acceleration through the OBD port or the CAN bus on these vehicles, we would be glad to receive it. It would let us re-check the electronic route before committing fully to physical actuation.

10. Estimated Cost of Physical Actuation

The following is a rough estimate for a first controlled accelerator prototype using physical actuation. These are approximate figures for planning only, and would be confirmed against actual GST invoices at the time of purchase.

Item	Approx. cost (INR)
12V linear actuator with position feedback (50 to 100mm stroke, about 150N)	2,500 to 5,000
Motor driver (BTS7960)	400 to 700
12V supply, fused tap and buck converter	1,000 to 2,000
Mounting bracket and fabrication to hold it against the pedal	500 to 1,500
Emergency kill switch and wiring	300 to 600
Microcontroller (ESP32)	Already in stock
Estimated total for a first accelerator prototype	about 5,000 to 10,000

Brake actuation is a separate and later task. It needs much greater force and stricter safety design, and is estimated higher, in the region of 8,000 to 15,000 rupees or more. We are keeping it in a later phase and are not ordering brake parts yet.

All actuation testing is planned for an open area under company supervision, with the car in neutral and the handbrake applied to begin with, and with hardware and software kill switches in place before anything is allowed to drive a pedal.

11. Project Repository

All the programs used in this work, including the acquisition script, the packet decoder and the receiver programs, are kept in the team's GitHub repository. It shows how the system is built and how each part works:

<https://github.com/loT-Internship-CarAutomation-LUS/Car-Automation>

12. Summary and Next Steps

Today's session proved the complete Track 1 data collection pipeline working on a real vehicle. Live engine data over Bluetooth and live GPS position were captured together on the Raspberry Pi, packed into structured 32-byte packets, logged locally, and forwarded over the network to a separate machine that decoded and verified every packet, with the same path ready to send to the cloud. In total 285 packets were captured with a 100 percent integrity pass rate.

The next steps are, first, to repeat the capture with the vehicle in motion so that changing speed and position can be recorded. Second, to use CAN sniffing to identify and add the remaining signals, such as brake, clutch and air-conditioning status, into the packet over the coming week. Third, for acceleration, to proceed with physical actuation of the accelerator pedal, subject to approval of the estimated parts cost above, while remaining open to any electronic-route material the team can share.